

A PROJECT REPORT

On

**EXECTRAK PRO – FIELD TRACKING SYSTEM FOR SALES
EXECUTIVES**

Done at

Tzahu Private Limited

By

Mr. Christy John

Reg. No.: LC24MCA011

Submitted to



DHONI P.O., PALAKKAD, KERALA – 678009

(Affiliated to the University of Calicut)

In partial fulfilment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

UNDER THE GUIDANCE OF

Dr. N RAVIA SHABNAM PARVEEN

Associate Professor & Head,

Department of Computer Applications

JUNE 2026

LEAD COLLEGE (AUTONOMOUS)
DHONI PO, OLAVAKKODE, PALAKKAD,
678009

CERTIFICATE

This is to certify that the project report titled, "ExecTrack Pro – Field Tracking System for Sales Executives", submitted by Mr. Christy John, Reg. No. LC24MCA011, a MCA student of LEAD College (Autonomous), is a bona fide work carried out by her in partial fulfilment of the requirements for the award of degree of Master of Computer Application of the University of Calicut.

Place: Palakkad

Date: _____

Dr. N Ravia Shabnam Parveen
Project Guide

Dr. Sreekanth Sreedharan
Director



Date : 13/06/2026

Name : Christy John

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Christy John employee of Tzahu Pvt Ltd from 20/04/2026 to 13/06/2026 and his employment details are as below :

Job Title : IT Sales Engineer

Department - Software Sales

We thank him for his dedication during his tenure with Tzahu Pvt Ltd and wish him every success in his professional journey ahead.

For Tzahu Pvt Ltd

Ashley Varghese
Chief Operations Officer

LEAD COLLEGE (AUTONOMOUS)
DHONI PO, OLAVAKKODE, PALAKKAD,
678009

DECLARATION

I, Mr. Christy John, Reg. No. LC24MCA011, hereby declare that this report titled "ExecTrack Pro – Field Tracking System for Sales Executives" is a record of my original work done under the guidance of Dr. N Ravia Shabnam Parveen, Associate Professor & HoD, Department of Computer Applications LEAD College (Autonomous), Dhoni, Palakkad. I further declare that this exposition is a result of my own effort and has not been submitted earlier for the award of any degree / diploma by the University of Calicut or any other University or any Academic body.

Place: Palakkad

Date: _____

Mr. Christy John

Reg. No. LC24MCA011

ACKNOWLEDGEMENT

I sincerely thank all those who have contributed, directly or indirectly, to the successful completion of this project.

*I extend my deep sense of gratitude to **Dr. Sreekanth Sreedharan**, Director, LEAD College (Autonomous), for granting me the permission and providing the necessary facilities to undertake this project. I also express my sincere thanks to **Prof. Rajkishan S. S.**, Deputy Director, LEAD College (Autonomous), for his constant support and encouragement throughout the project work.*

*I am highly indebted to **Dr. N Ravia Shabnam Parveen**, Associate Professor & HoD, Department of Computer Applications, LEAD College (Autonomous), for her valuable guidance, constant supervision, and constructive suggestions throughout the course of this project.*

*My profound sense of gratitude to **Mr. Sidharath V**, IT Head, Tzahu Private Limited, for graciously granting me the opportunity to undertake this project and for his invaluable cooperation, continuous support, and insightful guidance throughout the development and successful execution of the system..*

I would like to express my sincere appreciation to all the faculty members, library staff, and non-teaching staff of LEAD College (Autonomous), Palakkad, for their cooperation and timely assistance.

Finally, I express my heartfelt gratitude to my parents and friends for their constant support, encouragement, and cooperation, which greatly contributed to the successful completion of this project.

Christy John

ABSTRACT

ExecTrack Pro – Field Tracking System for Sales Executives is a mobile-based workforce management platform developed for Tzahu Private Limited to improve the monitoring and management of field sales operations. The project addresses challenges associated with traditional field workforce management practices, such as manual attendance recording, delayed reporting, limited visibility into employee activities, and difficulties in tracking customer interactions. These limitations often lead to inefficiencies, reduced transparency, and increased administrative effort.

The proposed system provides a centralized digital solution that enables organizations to efficiently manage field employees and monitor their activities in real time. The application is developed using Flutter and Dart for the frontend, while Python and Django REST Framework are used for backend development. The system is hosted on Amazon Web Services (AWS) and utilizes REST APIs for seamless communication between the mobile application and backend services. Secure authentication and role-based access control are implemented using JSON Web Tokens (JWT) to ensure data security and controlled access.

The system includes key modules such as Authentication Management, Attendance Management, Customer Visit Management, Activity Reporting, Location Tracking, Dashboard and Reporting, and Performance Evaluation. Field sales executives can mark attendance, record customer visits, submit activity updates, and share location information through the mobile application. Managers and administrators can access a centralized dashboard to monitor employee activities, review reports, and evaluate performance based on operational data and key performance indicators.

The proposed solution is expected to significantly reduce manual reporting efforts, improve operational transparency, and enhance workforce productivity. By providing real-time access to attendance records, customer interactions, and employee activities, the system supports informed decision-making and effective field workforce management. Furthermore, the scalable architecture of the application supports future enhancements such as CRM integration, advanced analytics, route optimization, and AI-driven performance monitoring.

<u>TABLE OF CONTENTS</u>		
Chapter	Section / Description	Page No.
Chapter 1	INTRODUCTION	
	1.1 Background	1
	1.2 Objectives	2
	1.3 Purpose, Scope, and Applicability	3
	1.4 Achievements	4
	1.5 Organization of Report	5
Chapter 2	SURVEY OF TECHNOLOGIES / LITERATURE REVIEW	
	2.1 Survey of Technologies	7
Chapter 3	REQUIREMENTS AND ANALYSIS	
	3.1 Problem Definition	12
	3.2 Requirements Specification	13
	3.3 Planning and Scheduling	15
	3.4 Software and Hardware Requirements	15
	3.5 Preliminary Product Description	16
	3.6 Conceptual Models	16
Chapter 4	SYSTEM DESIGN	
	4.1 Basic Modules – System Flow Design	20
	4.1.1 Input & Output / UI Design	21
	4.1.2 Data Flow Design	22
	4.1.3 Process Design	23
	4.1.4 Integration Design	25
	4.1.5 Implementation & Security Design	26
	4.2 Database design	26
Chapter 5	IMPLEMENTATION AND TESTING	
	5.1 Implementation Approaches	29
	5.2 Coding Details and Code Efficiency	30
	5.3 Testing Approach	31
Chapter 6	RESULTS AND DISCUSSION	
	6.1 Test Reports	35
	6.2 Coding	36
	6.3 Screen Shots	37
Chapter 7	CONCLUSIONS	
	7.1 Conclusion	43
	7.2 Limitations of the System	44
	7.3 Future Scope of the Project	45
	BIBLIOGRAPHY	46

LIST OF TABLES

Table No.	Caption	Page No.
Table 1	Technology Stack for ExecTrack Pro	10
Table 2	Software and Hardware Requirements	15
Table 3	Database Schema – Core Tables	27
Table 4	API Endpoint Specification	32
Table 5	Unit Test Results – Backend	35
Table 6	Testing Strategy and Coverage Matrix	36
Table 7	Expected Business Benefits	41

LIST OF FIGURES

Figure No.	Caption	Page No.
Fig. 1	<i>System Architecture Overview of ExecTech Pro</i>	17
Fig. 2	<i>Use Case Diagram – Field Sales Executive Module</i>	17
Fig. 3	<i>Use Case Diagram –Manager / Administrator Module</i>	18
Fig. 4	<i>Entity Relationship (ER) Diagram</i>	18
Fig. 5	<i>Data Flow Diagram – Level 0 (Context Diagram)</i>	22
Fig. 6	<i>Data Flow Diagram – Level 1</i>	23
Fig. 7	<i>Process Flow –ExecTrack Pro</i>	24
Fig. 8	<i>Integration Design</i>	25
Fig. 9	<i>Secure login with role-based redirection.</i>	37
Fig. 10	<i>Task dispatch with map-based location picker and worldwide place search.</i>	38
Fig. 11	<i>Real-time Live Map - avatar markers, searchable staff finder, and the live worker count.</i>	38
Fig. 12	<i>Active task - the worker's live position and the road route with ETA to the destination</i>	39
Fig. 13	<i>Team management - invite, edit, reset password, change role, and remove members</i>	39
Fig. 14	<i>Self-service profile and password management.</i>	40
Fig. 15	<i>Platform owner portal - tenants, seats, and MRR.</i>	40
Fig. 16	<i>Per-staff performance analytics.</i>	41

CHAPTER ONE

INTRODUCTION

INTRODUCTION

1.1 Background

The rapid expansion of sales-driven organizations has significantly increased their dependence on field sales teams to generate revenue, engage clients, and sustain business relationships. Managing a geographically dispersed workforce, however, presents formidable operational challenges. Without real-time information about the location, activities, and performance of field staff, managers are often left relying on delayed phone calls, end-of-day email reports, and manually maintained spreadsheets to understand what is happening in the field. These traditional approaches are not only inefficient but also prone to data inaccuracies, reporting gaps, and coordination failures that adversely impact organizational productivity.

Tzahu Private Limited is a Palakkad-based company offering Sales and Marketing Services, Application Development, and CRM Solutions to its business clients. The company employs a team of field sales executives who are deployed across various client sites to conduct customer visits, record interactions, and report daily activities. Prior to this project, the company relied on a combination of manual attendance registers, WhatsApp reporting, and periodic telephonic check-ins to monitor its field workforce. This methodology resulted in significant delays in data availability, inconsistencies in attendance and visit records, and limited managerial visibility into employee productivity and performance trends.

The ExecTrack Pro project was conceived to address precisely these challenges. Developed as an MCA final semester project in collaboration with Tzahu Private Limited, ExecTrack Pro is a mobile-first field-work management system that replaces manual, paper-based processes with a centralized, real-time digital platform. The system integrates a Flutter-based mobile application for field executives with a Django REST Framework backend, cloud hosting on Amazon Web Services, and geolocation services via the Google Maps API, thereby providing a holistic solution for workforce monitoring, attendance management, customer visit tracking, and performance evaluation.

1.2 Objectives

The primary and secondary objectives of the ExecTrack Pro system are defined as follows:

Primary Objectives:

- To develop a centralized field-work platform combining a Flutter mobile application and a Django REST API backend.
- To automate attendance management through GPS-enabled digital check-in and check-out functionality.
- To enable systematic digital tracking of customer visits, including remarks, photographs, and follow-up scheduling.
- To maintain comprehensive employee activity logs that serve as the basis for daily operational reporting.
- To provide real-time location monitoring of field executives during working hours, with appropriate consent mechanisms.
- To facilitate two-way communication through push notifications and in-app messaging between field staff and management.
- To generate performance reports and analytics dashboards covering attendance compliance, visit frequency, and key performance indicators.

Secondary Objectives:

- To reduce manual reporting effort within the organization by a minimum of 80%.
- To improve operational transparency by achieving at least 95% capture of field activities in the system.
- To enhance employee accountability through audit-logged activity records and location-based verification.
- To deploy the system on a scalable cloud-based infrastructure suitable for future growth.
- To establish a foundation for future enhancements such as CRM integration, AI-driven insights, and advanced geofencing capabilities.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of ExecTrack Pro is to digitize and streamline the end-to-end management of field sales operations for Tzahu Private Limited. By providing a unified platform for attendance recording, customer visit logging, activity reporting, and performance evaluation, the system eliminates the inefficiencies inherent in manual and telephone-based field management. The platform serves as a single source of truth for all field activity data, enabling managers to make informed, timely decisions and enabling executives to manage their daily workload more efficiently.

1.3.2 Scope

The scope of the ExecTrack Pro system encompasses the following functional boundaries. The mobile application, developed using Flutter, targets Android devices (with provisions for iOS compatibility) and provides field executives with tools for authentication, attendance marking, customer visit logging, activity submission, and GPS location sharing. The backend API, built on Django REST Framework, handles all business logic, data persistence, authentication, and report generation. The system supports three user roles — Field Sales Executive, Manager, and Administrator — each with a distinct set of permissions and capabilities. Geographic functionality is limited to GPS coordinate capture and Google Maps-based location display; advanced geofencing features are identified as a future enhancement.

The present version of the system uses SQLite as the database for the prototype phase, with a documented migration path to PostgreSQL or Amazon Aurora for production deployment. Cloud hosting is provisioned through Amazon Elastic Beanstalk and Amazon S3. Real-time push notifications are supported through the system's notification service module. The web-based management portal, identified in the project roadmap, falls outside the current implementation scope and is earmarked for the 2028 H2 release cycle.

1.3.3 Applicability

ExecTrack Pro is directly applicable to Tzahu Private Limited as the primary client organization. Beyond this immediate deployment, the system architecture and design are broadly applicable to any organization that manages a field workforce, including FMCG distribution companies, pharmaceutical sales organizations, insurance agencies,

logistics and delivery firms, and facility management companies. The multi-role access control model and modular architecture ensure that the system can be adapted to different organizational structures with minimal reconfiguration. The cloud deployment model further ensures that the system can scale to accommodate organizations of varying sizes, from small teams of ten executives to large enterprises with hundreds of field staff.

1.4 Achievements

The ExecTrack Pro project achieved the following outcomes within the defined development timeline:

- Successful development and deployment of a fully functional Flutter mobile application supporting attendance management, customer visit logging, activity reporting, and GPS location sharing.
- Implementation of a secure Django REST Framework backend with JWT-based authentication, role-based access control, and a comprehensive set of CRUD APIs covering all system modules.
- Completion of a working database schema with nine core tables covering users, attendance, customers, visits, activities, locations, performance scores, notifications, and audit logs.
- Development of a dashboard and reporting module enabling managers to generate, view, and export attendance and performance reports in CSV and JSON formats.
- Integration of Google Maps API for GPS coordinate capture and reverse geocoding during check-in, check-out, and visit logging operations.
- Implementation of a CI/CD pipeline using GitHub Actions and AWS CodeDeploy for automated testing and deployment.
- Achievement of greater than 90% line coverage in backend unit tests using pytest, with load testing confirming system stability under 200 concurrent users at an average response time below one second.
- Positive user acceptance testing outcomes from a pilot group of ten field executives, with reported satisfaction rates meeting the target threshold.

1.5 Organization of Report

This report is organized into seven chapters, each addressing a distinct phase of the project lifecycle. Chapter 1 provides an introduction to the project, including its background, objectives, scope, and summary of achievements. Chapter 2 presents a survey of the technologies employed in the system and reviews relevant literature in the field of mobile workforce management and field tracking systems. Chapter 3 details the requirements analysis, including the problem definition, functional and non-functional requirements, planning and scheduling, software and hardware specifications, and conceptual models. Chapter 4 describes the system design through module flow diagrams, data flow diagrams, process design, integration architecture, and security design. Chapter 5 covers implementation and testing, documenting the development approach, key coding components, and the testing strategy including unit and integration testing. Chapter 6 presents the results and discussion, comprising test reports, code descriptions, and application screenshots. Chapter 7 concludes the report with a summary of findings, system limitations, and the identified future scope. The report also includes a bibliography of all references cited and an annexure containing additional supporting materials.

CHAPTER TWO

SURVEY OF TECHNOLOGIES

SURVEY OF TECHNOLOGIES

2.1 Survey of Technologies

The development of ExecTrack Pro required the careful selection and integration of several modern technologies across the mobile frontend, backend API, database, cloud infrastructure, and third-party services layers. This section provides a comprehensive technical survey of each technology employed, analyzing its characteristics, the rationale for its selection, and its specific role within the system.

2.1.1 Flutter and Dart

Flutter is an open-source UI toolkit developed by Google that enables the creation of natively compiled, cross-platform applications from a single codebase [1]. Built on the Dart programming language, Flutter employs a widget-based rendering architecture in which every visual element of the user interface is a widget, from structural containers to interactive buttons and text inputs. Flutter compiles to native ARM code for both Android and iOS platforms, delivering high-performance graphical rendering at 60 frames per second without relying on a JavaScript bridge, which is the common bottleneck in hybrid frameworks such as React Native or Ionic.

For ExecTrack Pro, Flutter was selected as the mobile frontend technology for three primary reasons. First, its cross-platform capability allows the application to target both Android and iOS devices from a unified codebase, significantly reducing development time and maintenance overhead. Second, Flutter's rich widget library and support for custom painting provide the flexibility needed to implement complex UI elements such as GPS-enabled attendance forms, visit log entry screens with photo attachment capabilities, and dashboard charts. Third, Flutter's offline-first architecture support, particularly its compatibility with local caching libraries such as sqflite, is essential for ensuring that field executives can continue to record activities in areas with poor network connectivity, with data synchronizing automatically when connectivity is restored.

2.1.2 Django REST Framework

Django is a high-level Python web framework that promotes rapid development and clean, pragmatic design [2]. The Django REST Framework (DRF) extends Django's capabilities to provide a powerful and flexible toolkit for building Web APIs. DRF features include a serialization layer that converts complex Python data types to JSON and back, class-based views that map HTTP methods to create, retrieve, update, and delete operations, a browsable API interface for debugging, and built-in support for authentication, permissions, and throttling.

The selection of Django REST Framework for the ExecTrack Pro backend was guided by several technical considerations. Django's batteries-included philosophy provides out-of-the-box support for common web application concerns such as ORM-based database interaction, form validation, input sanitization, and administrative interface generation, which substantially reduces the amount of boilerplate code that developers must write. DRF's serializer framework enables clean separation between data validation and business logic, while its ViewSet and Router system allows the definition of complete CRUD APIs with minimal code repetition. The availability of the `djangorestframework-simplejwt` library for JSON Web Token authentication and the `drf-yasg` library for automatic Swagger UI documentation generation further reinforced the selection of this framework.

2.1.3 SQLite Database

SQLite is a serverless, file-based relational database management system that is widely used for embedded and prototype database scenarios [3]. Unlike client-server database systems such as MySQL or PostgreSQL, SQLite requires no separate server process; the entire database is stored in a single file on disk and accessed directly by the application process through standard SQL queries. This zero-configuration model makes SQLite extremely convenient for development and testing environments.

In ExecTrack Pro, SQLite serves as the prototype database during the development and initial deployment phases. The Django ORM abstracts the database layer, meaning that the application's data access code is entirely independent of the underlying database engine. This design decision ensures a straightforward migration path from SQLite to a production-grade relational database system such as PostgreSQL or Amazon Aurora when the system is scaled to handle larger data volumes and concurrent users. The

migration process requires only a change to the database connection string in the project's configuration file.

2.1.4 Amazon Web Services (AWS)

Amazon Web Services is the world's most comprehensive and broadly adopted cloud platform, offering over 200 fully featured services from data centers globally [4]. For ExecTrack Pro, two primary AWS services are employed: Elastic Beanstalk and Amazon Simple Storage Service (S3).

AWS Elastic Beanstalk is a Platform-as-a-Service (PaaS) offering that enables developers to deploy and scale web applications without managing the underlying infrastructure. Elastic Beanstalk automatically handles the deployment details of capacity provisioning, load balancing, auto-scaling, and application health monitoring. The deployment model for ExecTrack Pro uses Elastic Beanstalk to host the Dockerized Django application, with support for blue-green deployments that minimize downtime during application updates. Amazon S3 is used for the storage of static assets including employee visit photographs uploaded through the mobile application, report exports, and application media files. S3's 11 nines of durability, cost-effective pricing, and seamless integration with the Django storage backend through the `django-storages` library make it an appropriate choice for this purpose.

2.1.5 JSON Web Tokens (JWT)

JSON Web Token is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object [5]. A JWT consists of three parts: a header that specifies the algorithm used for signing, a payload that contains the claims or assertions about the user, and a cryptographic signature that verifies the token's integrity. In stateless API authentication, the server issues a JWT upon successful login, and the client includes this token in the Authorization header of all subsequent requests. The server validates the token's signature without querying a session store, enabling highly scalable authentication for mobile applications.

ExecTrack Pro implements JWT authentication through the `django-rest-framework-simplejwt` library. Access tokens are issued with a 15-minute expiry to minimize the window of exposure in the event of token theft, while refresh tokens are issued with a 7-day validity period, allowing clients to obtain new access tokens without requiring

the user to log in again. This dual-token strategy strikes a balance between security and usability appropriate for a mobile field application.

2.1.6 Google Maps API

The Google Maps API provides a suite of location-based services including geocoding, reverse geocoding, place search, and interactive map rendering [6]. In ExecTrack Pro, the Google Maps API serves two primary functions. First, it provides reverse geocoding services that convert raw GPS latitude and longitude coordinates captured during check-in, check-out, and customer visit events into human-readable address strings, which are stored alongside the coordinate data in the database and displayed in reports. Second, the Google Maps API is used to render interactive map views within the application for location visualization purposes.

Layer	Technology	Rationale
Mobile UI	Flutter (Dart)	Cross-platform, native performance, offline-first support
API Server	Django REST Framework	Rapid development, built-in admin, robust serialization
Database	SQLite (Prototype)	Zero-config MVP; migrates to PostgreSQL/Aurora
Cloud Hosting	AWS Elastic Beanstalk / ECS	Managed scaling, easy CI/CD integration
Static Assets	Amazon S3	Durable, cost-effective storage for images and PDFs
Authentication	JWT (simplejwt)	Stateless, mobile-friendly, scalable
Geolocation	Google Maps API	Accurate GPS, reverse geocoding, map rendering
CI/CD	GitHub Actions + AWS CodeDeploy	Automated build, test, and deployment pipeline
API Docs	Swagger / OpenAPI (drf-yasg)	Auto-generated interactive documentation

Table 1. Technology Stack for ExecTrack Pro

CHAPTER THREE

REQUIREMENTS AND ANALYSIS

REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

Tzahu Private Limited operates with a distributed field sales team responsible for visiting clients, recording customer interactions, and submitting daily activity summaries to management. Prior to the implementation of ExecTrack Pro, the company's field management processes were characterized by a number of structural inefficiencies that collectively impeded operational effectiveness and managerial decision-making.

The first and most significant problem was the absence of real-time visibility into field staff activities. Managers had no mechanism to verify whether a sales executive had visited a particular client, arrived at a scheduled location, or completed an assigned task at any given point during the working day. The only information available was whatever the executive chose to report, either verbally or via a daily summary submitted at the end of the workday.

The second problem concerned attendance recording. Manual attendance registers were maintained at the company office, which created a fundamental conflict: field executives who began their day directly from client sites had no means of recording their attendance at the office. This led to underreporting of productive time and inaccurate attendance data that was disconnected from actual field operations.

The third problem related to customer visit documentation. Visit records were maintained by individual executives on personal notebooks or mobile phone notes, with no standardized format, no photograph documentation, and no systematic mechanism for recording follow-up requirements. This fragmented record-keeping made it impossible for management to analyze visit patterns, identify high-priority clients, or track the progress of ongoing customer engagements.

The fourth problem was the lack of performance analytics. Without centralized, structured data on visits, attendance, and task completion, management had no quantitative basis for evaluating individual executive performance, identifying underperforming team members, or recognizing and rewarding top performers. Performance assessments were therefore entirely subjective, leading to employee dissatisfaction and inconsistent accountability.

Taken together, these problems created a significant operational gap: the organization was investing substantial resources in field sales activities while having minimal visibility into the effectiveness of those activities and no systematic mechanism for improvement. ExecTrack Pro was conceived to close this gap comprehensively.

3.2 Requirements Specification

The requirements for ExecTrack Pro were gathered through structured discussions with the company guide, Mr. Sidharath V, IT Head, Tzahu Private Limited, and supplemented by observation of existing manual workflows. The requirements are classified into functional and non-functional categories.

3.2.1 Functional Requirements

The system shall provide the following functional capabilities:

- **Authentication Management:** The system shall support secure user registration (by administrators), login with username and password, JWT-based session management, role-based access control for Executives, Managers, and Administrators, and password reset functionality.
- **Attendance Management:** The system shall enable field executives to digitally mark attendance check-in and check-out from their mobile devices, capturing GPS coordinates, timestamp, and location address at each event. The system shall maintain a complete attendance history and compute daily attendance status (present, absent, late).
- **Customer Visit Management:** The system shall allow executives to add customer records, log individual customer visits with date, time, remarks, and optional photograph attachments, and schedule follow-up visits. All visit records shall be stored centrally and accessible to managers.
- **Activity Reporting:** The system shall provide a daily activity submission feature enabling executives to document their tasks, observations, and progress notes. Submitted reports shall be viewable by managers in a consolidated format.
- **Location Tracking:** The system shall capture GPS coordinates during logged-in working hours, store location traces in the database, and present location data on map views for managers. Location tracking shall be explicitly consent-based and shall cease automatically upon check-out.

- **Dashboard and Reporting:** The system shall provide role-appropriate dashboards displaying attendance summaries, visit counts, activity status, and performance metrics. Managers and administrators shall be able to export attendance and performance reports in CSV and JSON formats.
- **Performance Evaluation:** The system shall compute performance scores for each executive based on configurable weights applied to attendance compliance, visit frequency, task completion rates, and customer engagement metrics.
- **Notification Service:** The system shall support push notification delivery to field executives for reminders, alerts, and management communications. Administrators shall be able to compose and send notifications through the application interface.

3.2.2 Non-Functional Requirements

- **Performance:** The system shall support a minimum of 200 concurrent users with average API response times not exceeding one second under normal load conditions.
- **Security:** All network communication shall use TLS 1.2 or higher. Passwords shall be hashed using PBKDF2 with per-user salts. All privileged actions shall be captured in an audit log.
- **Availability:** The system shall target 99.5% uptime, with nightly automated database backups to Amazon S3 with a 30-day retention policy.
- **Scalability:** The architecture shall be designed to support horizontal scaling through AWS Elastic Beanstalk auto-scaling groups, enabling the system to accommodate organizational growth without architectural changes.
- **Usability:** The mobile application interface shall be intuitive enough for field executives with basic smartphone proficiency to use without formal training.
- **Maintainability:** The codebase shall be structured according to Django's application convention and Flutter's feature-first directory structure, with inline documentation and auto-generated Swagger API documentation.

3.3 Planning and Scheduling

The ExecTrack Pro project was executed over an eight-week development period, from April 5, 2026 to June 5, 2026, following an iterative development methodology with defined phase milestones. The project was structured into five phases: requirements gathering and system design, backend API development, mobile application development, integration and testing, and deployment and documentation.

The requirements gathering and system design phase occupied the first two weeks, during which stakeholder discussions were conducted, wireframes and UI mockups were created, the database schema was finalized, and the API specification was drafted. Backend API development was undertaken in weeks three and four, covering all eight functional modules including authentication, attendance, visit management, activity reporting, location tracking, reporting, performance evaluation, and notifications. Mobile application development proceeded in parallel during weeks three through five, with the Flutter screens and state management logic aligned to the finalized API specification.

Integration and testing occupied weeks five and six, during which the mobile application was connected to the live backend API, end-to-end test cases were executed, and identified bugs were resolved. Weeks seven and eight were dedicated to deployment configuration on AWS, user acceptance testing with the pilot group, and the preparation of project documentation including this report.

3.4 Software and Hardware Requirements

Category	Development Environment	Deployment Environment
Operating System	Windows 10 / Ubuntu 22.04 LTS	Ubuntu 22.04 LTS (AWS EC2)
Programming Language	Python 3.11, Dart 3.x	Python 3.11, Dart 3.x
Backend Framework	Django 4.2, DRF 3.14	Django 4.2, DRF 3.14
Mobile Framework	Flutter 3.19 SDK	Flutter 3.19 SDK (Build)
Database	SQLite 3 (built-in)	SQLite 3 / PostgreSQL 15
IDE	VS Code, Android Studio	–
Version Control	Git / GitHub	GitHub Actions (CI/CD)

Mobile Device	Android 10+ (emulator/physical)	Android 10+ (end-user)
Cloud Platform	–	AWS Elastic Beanstalk, S3
RAM	Minimum 8 GB (development)	Minimum 2 GB (EC2 t3.small)
Storage	50 GB (development)	20 GB EBS + S3 (unlimited)

Table 2. Software and Hardware Requirements

3.5 Preliminary Product Description

ExecTrack Pro is a mobile-first field-work management system comprising two primary components: a Flutter mobile application and a Django REST API backend. The mobile application serves as the primary interface for field sales executives and provides screens for authentication, attendance check-in and check-out, customer visit logging, daily activity submission, and personal report viewing. The backend API processes all client requests, enforces business rules, manages data persistence in the SQLite database, and exposes RESTful endpoints conforming to the OpenAPI 3.0 specification.

The system supports three user roles with distinct interfaces and permissions. Field Sales Executives access the mobile application exclusively and are permitted to perform operations on their own data only. Managers access a consolidated dashboard showing team-wide attendance, visit, and activity data, with report generation and export capabilities. Administrators have full system access including user management, configuration, and security audit log review.

The preliminary product includes the following quantifiable characteristics: nine database tables, ten core API endpoint groups, eight functional application modules, three user roles with distinct permission sets, integration with two external services (Google Maps API and AWS), and a CI/CD pipeline covering automated unit testing, Docker image building, and Elastic Beanstalk deployment.

3.6 Conceptual Models

The conceptual design of ExecTrack Pro is articulated through a set of standardized models that capture the system's user interactions, data relationships, and operational flows.

3.6.1 System Architecture

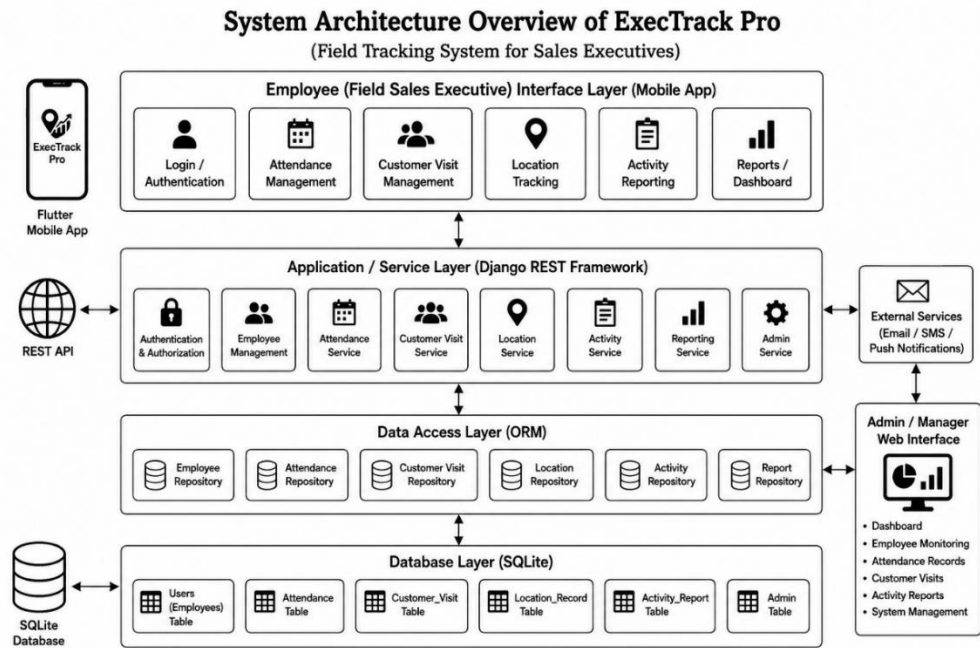


Fig. 1. System Architecture Overview of ExecTech Pro

3.6.2 Use Case Model

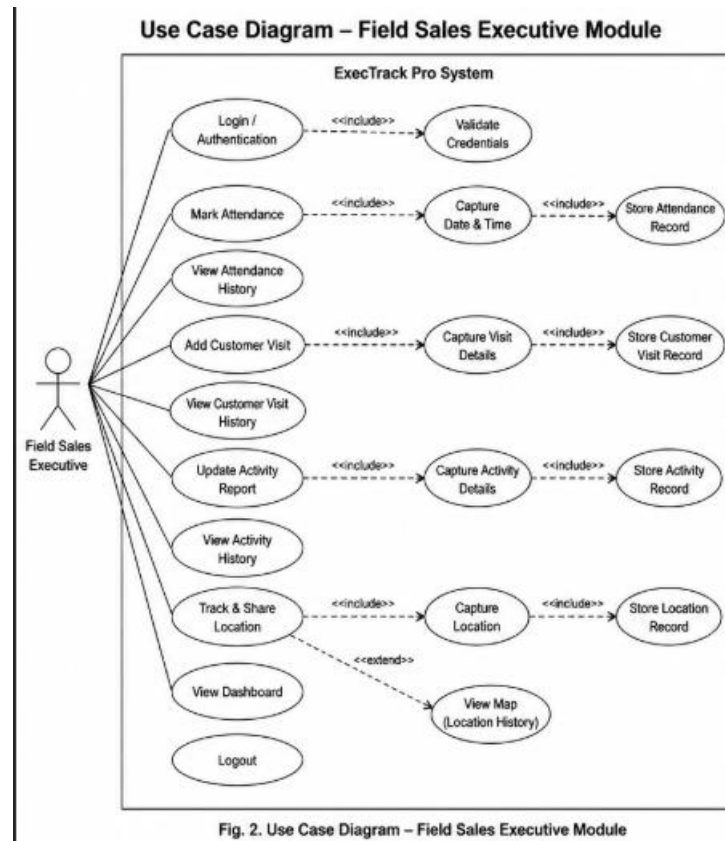


Fig. 2. Use Case Diagram – Field Sales Executive Module

Fig. 2. Use Case Diagram – Field Sales Executive Module

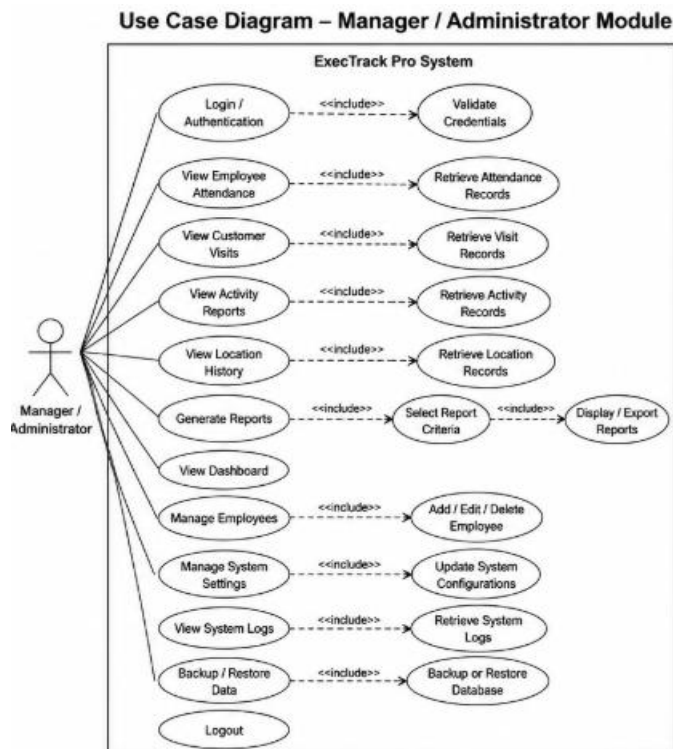


Fig. 3. Use Case Diagram – Manager / Administrator Module

Fig. 3. Use Case Diagram –Manager / Administrator Module

3.6.3 Entity Relationship Diagram

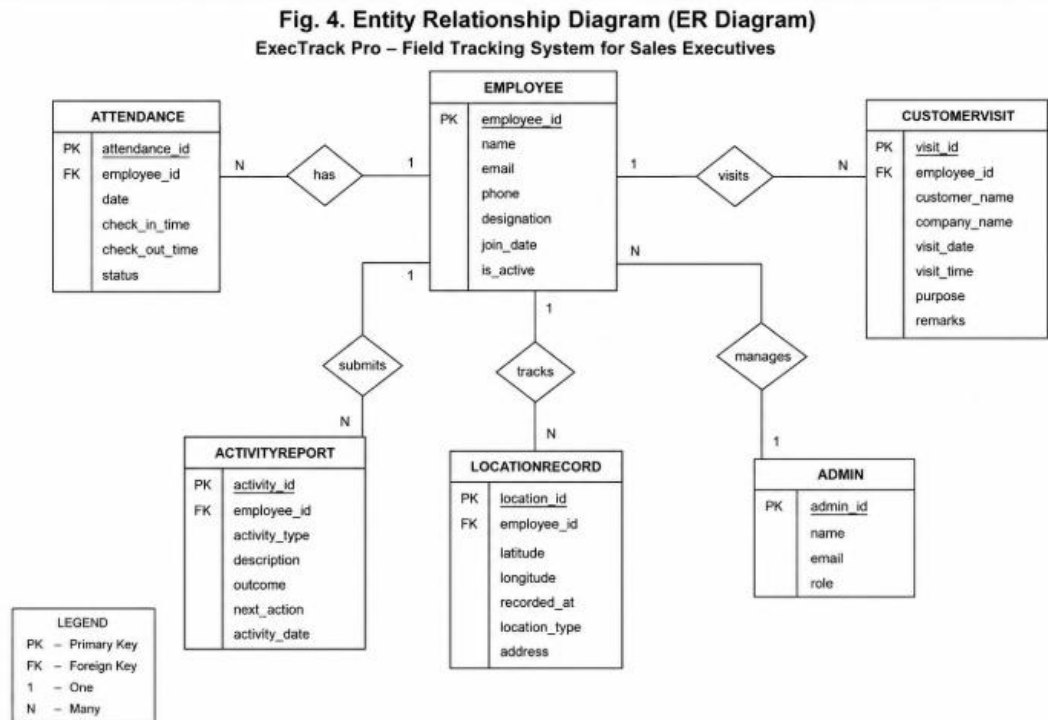


Fig. 4. Entity Relationship (ER) Diagram

CHAPTER FOUR

SYSTEM DESIGN

SYSTEM DESIGN

System design translates the requirements and conceptual models developed in Chapter 3 into a concrete architectural blueprint that guides implementation. This chapter presents the comprehensive design of ExecTrack Pro across five dimensions: basic module and system flow design, input and output or user interface design, data flow design, process design, and integration and security design.

4.1 Basic Modules – System Flow Design

ExecTrack Pro is organized into eight functional modules, each encapsulating a distinct domain of system behavior. The modular architecture ensures separation of concerns, facilitates independent testing, and supports future extension without disrupting existing functionality.

The Authentication Management module is the entry point for all users and is responsible for registration (administrator-initiated), login, JWT token issuance and refresh, password reset, and role-based access enforcement. It serves as the security perimeter through which all other modules are accessed.

The Attendance Management module handles the core operational task of recording the daily working hours of field executives. It captures GPS coordinates, timestamps, and device metadata at check-in and check-out events, stores this data in the attendance table, and computes daily attendance status by comparing check-in time against a configurable late-arrival threshold.

The Customer Visit Management module maintains a master list of customer records and logs individual visit events against those records. Each visit entry captures the visiting executive's identity, the customer visited, visit timestamp, textual remarks, an optional photograph, and a follow-up date. The module provides list and detail views for managers to review visit histories by executive, by customer, or by date range.

The Activity Reporting module provides a structured daily reporting mechanism. Executives submit end-of-day summaries documenting their tasks, accomplishments, and observations in a structured JSON format. These reports feed into the managerial dashboard and contribute to the performance evaluation module's scoring calculations.

The Location Tracking module collects GPS coordinates at regular intervals during active working hours and stores them in the locations table. It provides a trail of location data that can be visualized on a map to verify field executive movements and confirm visit accuracy.

The Dashboard and Reporting module aggregates data from the attendance, visits, activities, and performance tables to produce role-appropriate summary views. Managers can generate attendance reports, visit frequency analyses, and individual performance summaries, and can export these reports in CSV format for use in external tools.

The Performance Evaluation module computes a composite performance score for each executive over configurable periods. The score is derived from a weighted combination of attendance compliance percentage, number of customer visits completed, activity submission rate, and a qualitative engagement factor derived from manager ratings. Scores are stored in the `performance_scores` table and displayed in the managerial dashboard.

The Notification Service module enables administrators to compose and dispatch push notifications to individual executives or groups. It stores notification metadata including delivery timestamps and read receipts in the notifications table, supporting communication about schedule changes, task assignments, and performance feedback.

4.1.1 Input and Output / UI Design

The user interface of ExecTrack Pro's mobile application is designed around the principles of simplicity, efficiency, and contextual appropriateness for field use. The application employs a bottom navigation bar architecture with four primary navigation destinations: Home (Dashboard), Attendance, Visits, and Profile. Each destination contains purpose-specific screens with minimal input requirements to accommodate usage in challenging field environments such as outdoor locations, moving vehicles, and areas with limited lighting.

The Login Screen presents two text fields for username and password, a Login button, and a Forgot Password link. Upon successful authentication, the user is directed to the role-appropriate home screen. The Attendance Screen presents a prominent Check-In button when the executive has not yet checked in for the day, and a Check-Out button

once check-in has been recorded. Both buttons display a loading indicator while the GPS coordinate is being captured and the API request is being processed.

The Visit Log Screen presents a form with fields for customer selection (from a searchable dropdown populated from the customer table), visit date and time (defaulting to the current date and time), remarks text area, and an optional photograph attachment accessed through the device camera or gallery. The Daily Activity Screen presents a structured form for task documentation with a summary text field, a task list section where executives can add individual tasks with completion status, and a notes section for additional observations.

4.1.2 Data Flow Design

The data flow architecture of ExecTrack Pro follows a client-server model in which all data originates at the Flutter mobile client, is transmitted over HTTPS to the Django REST API backend, processed and persisted in the SQLite database, and returned to authorized clients in JSON format for rendering in the user interface.

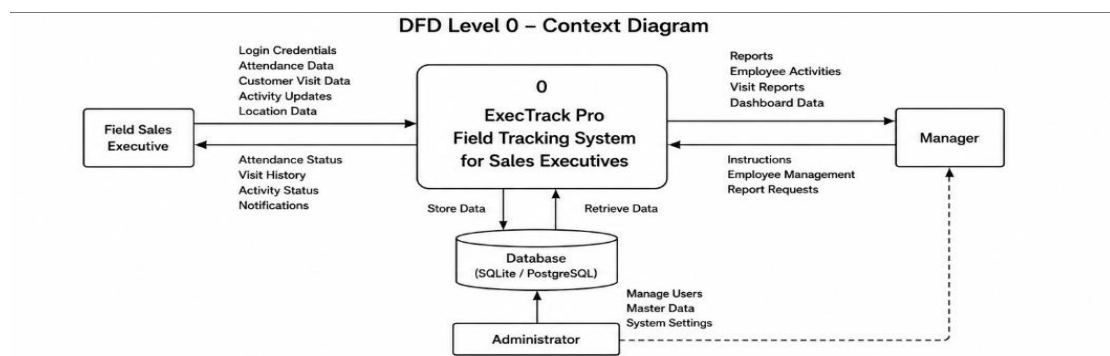


Fig. 5. Data Flow Diagram – Level 0 (Context Diagram)

The Level 0 Context Diagram presented in Figure 5 illustrates the ExecTrack Pro system as a single process entity receiving data flows from three external entities: the Field Sales Executive, the Manager, and the Administrator. The Executive inputs attendance events, visit records, activity reports, and location data into the system. The Manager inputs performance evaluation parameters and notification content and receives attendance reports, visit summaries, performance scores, and location data. The Administrator inputs user configuration and system settings and receives audit logs and system health data.

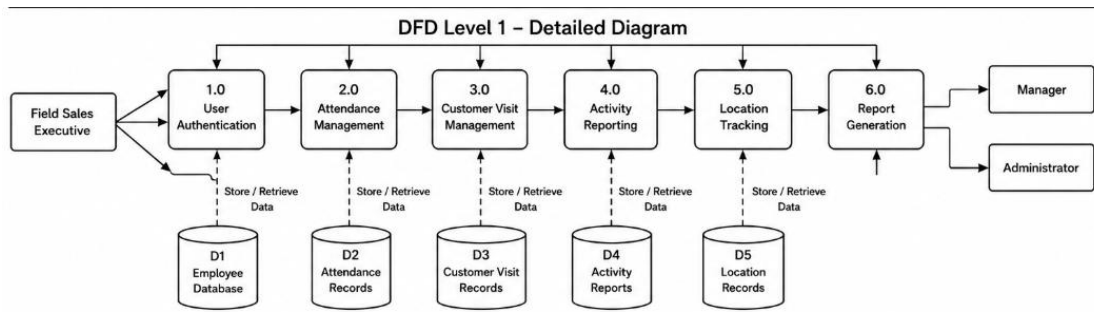


Fig. 6. Data Flow Diagram – Level 1

The Level 1 Data Flow Diagram presented in Figure 6 decomposes the system process into its eight functional modules, showing the specific data stores accessed and updated by each module and the data flows between modules. The Authentication module accesses the users data store to validate credentials and issues tokens that flow to all other modules as authorization inputs. The Attendance module writes to the attendance data store and reads from the users data store. The Visit module writes to the visits and customers data stores. The Activity module writes to the activities data store. The Location module writes to the locations data store. The Reporting module reads from the attendance, visits, activities, and performance_scores data stores to produce report outputs. The Performance module reads from all operational data stores and writes computed scores to the performance_scores data store. The Notification module writes to and reads from the notifications data store.

4.1.3 Process Design

The process design of ExecTrack Pro defines the algorithmic logic governing each functional module's core operations. The attendance check-in process is illustrative of the system's process design philosophy. When a field executive initiates a check-in from the mobile application, the application first verifies that the device's GPS service is enabled and requests location permission if not already granted. The GPS coordinate is then captured with an accuracy threshold of 50 meters; if the captured accuracy is lower than this threshold, the application retries up to three times before proceeding with the best available coordinate. The coordinate, along with the current timestamp and the executive's JWT identity, is transmitted via a POST request to the /api/attendance/checkin endpoint. The backend validates the JWT, checks that no active check-in record exists for this executive on the current date, creates a new attendance record with status PRESENT, invokes the Google Maps reverse geocoding API to

obtain a human-readable address from the coordinate, updates the attendance record with the address, and returns a success response to the client. If the device is offline at the time of check-in, the request is queued in the application's local SQLite cache and transmitted automatically when connectivity is restored.

A similar process governs customer visit logging, activity submission, and all other write operations in the system. The key invariant maintained across all processes is that the backend is the single authoritative source of truth; the mobile client serves as an interface and an offline buffer, but all data validation and business rule enforcement occurs server-side.

**Process Flow Diagram – ExecTrack Pro
(Field Tracking System for Sales Executives)**

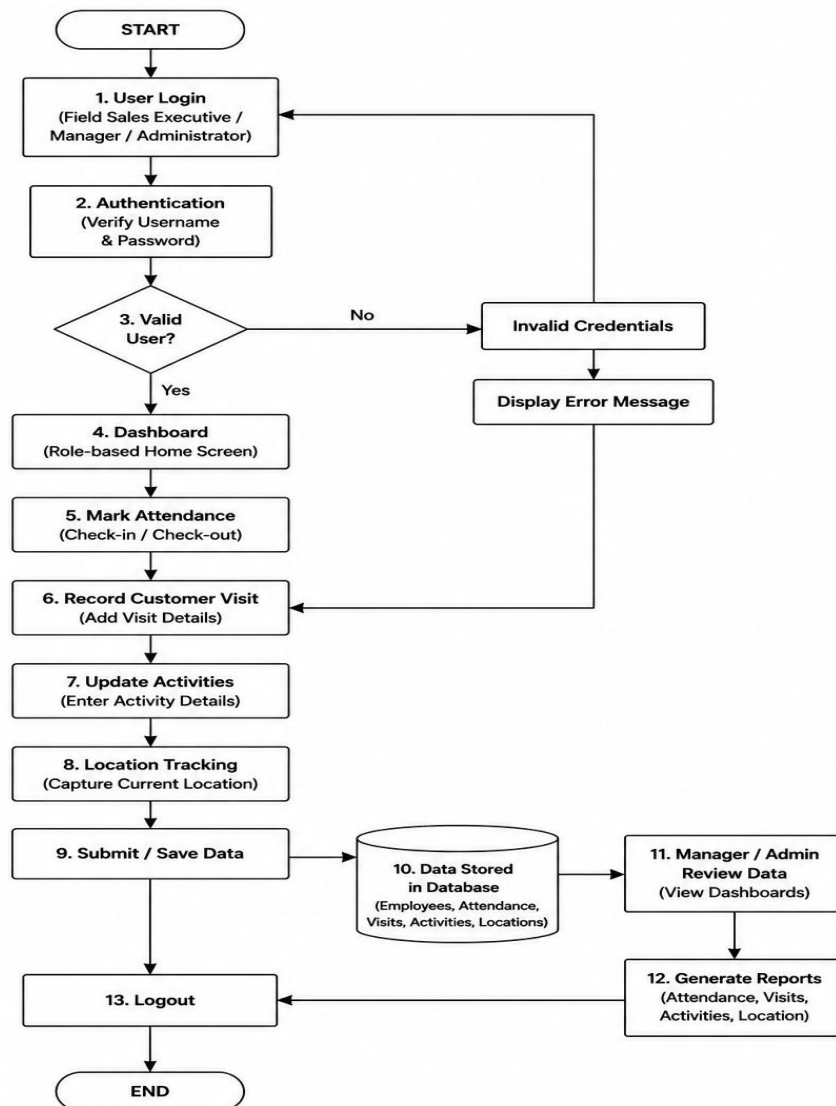


Fig. 7. Process Flow –ExecTrack Pro

4.1.4 INTEGRATION DESIGN

The integration design of ExecTrack Pro ensures seamless communication between the mobile application, backend services, database, cloud services, and external services. All modules are integrated through REST APIs to provide a secure, scalable, and efficient system.

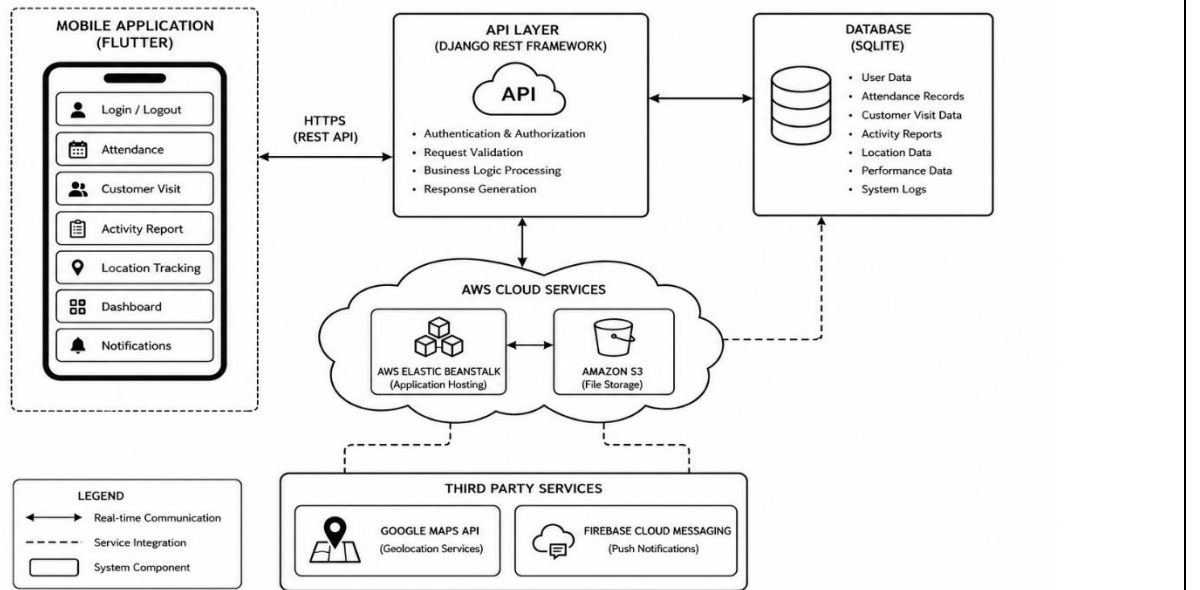


Fig. 8. Integration Design

ExecTrack Pro integrates with two external services — the Google Maps API and Amazon Web Services — and maintains internal integration between the Flutter mobile client and the Django REST backend through a well-defined API contract documented in the OpenAPI 3.0 Swagger specification.

The Google Maps API integration is implemented at the backend level through the geopy library and direct HTTP requests to the Google Geocoding API endpoint. This server-side integration approach, as opposed to a client-side approach, ensures that the Google API key is never exposed in the mobile application binary, protecting against potential misuse. API calls to Google Maps are rate-limited and cached at the server level to minimize cost and latency.

The AWS integration encompasses two components. The Elastic Beanstalk integration is handled through the AWS CodeDeploy GitHub Actions workflow, which packages the Dockerized Django application, pushes the image to Amazon ECR, and triggers a blue-green deployment on the Elastic Beanstalk environment. The S3 integration is implemented through the django-storages library with the S3Boto3Storage backend, which transparently redirects all media file uploads — specifically visit photographs — to the configured S3 bucket.

4.1.5 Implementation and Security Design

The security design of ExecTrack Pro is implemented across multiple layers of the system stack, following the principle of defence in depth. At the network layer, all communication between the Flutter client and the Django backend is encrypted using TLS 1.2 or higher, enforced through the AWS Elastic Beanstalk HTTPS listener configuration.

At the authentication layer, JWT access tokens are issued with a 15-minute expiry, minimizing the window of opportunity for token replay attacks. Refresh tokens with a 7-day validity are stored securely in the mobile application's encrypted shared preferences storage, preventing extraction by other applications on the device. The `djangorestframework-simplejwt` library implements token blacklisting to invalidate refresh tokens upon logout, preventing token reuse after session termination.

At the application layer, role-based access control is enforced through DRF permission classes applied to every API endpoint. The `IsAuthenticated` permission class ensures that all protected endpoints require a valid JWT. Custom permission classes (`IsManagerOrAdmin`, `IsAdminOnly`) restrict access to sensitive endpoints such as report generation and user management. Input validation is enforced through DRF serializer classes that define allowed field types, maximum lengths, and value constraints, protecting against injection attacks and malformed data.

At the data layer, all passwords are hashed using Django's default PBKDF2-SHA256 algorithm with a per-user random salt, ensuring that password database compromises do not expose plaintext credentials. All privileged actions — including login events, data exports, configuration changes, and user management operations — are recorded in the `audit_log` table with the acting user's identity, IP address, action type, and timestamp, providing a complete security audit trail.

4.2 Database Design

The database schema for ExecTrack Pro consists of nine tables, designed to support all functional modules while maintaining referential integrity and query performance. The schema follows third normal form principles to minimize data redundancy and ensure data consistency.

Table	Key Columns	Description
users	id (PK), username, email, password_hash, role (enum), created_at	User authentication, role, and profile information
attendance	id (PK), user_id (FK), check_in_ts, check_out_ts, lat_in, lon_in, lat_out, lon_out, status (enum)	Daily check-in and check-out records with GPS coordinates
customers	id (PK), name, contact, address, created_by (FK)	Master customer list maintained by executives and admins
visits	id (PK), user_id (FK), customer_id (FK), visit_ts, remarks, photo_path, follow_up_date	Individual customer visit event records
activities	id (PK), user_id (FK), date, summary, tasks_json, notes	Daily activity logs submitted by field executives
locations	id (PK), user_id (FK), timestamp, lat, lon, accuracy	Raw GPS trace data captured during working hours
performance_scores	id (PK), user_id (FK), period_start, period_end, score, details_json	Computed KPI scores per executive per evaluation period
notifications	id (PK), user_id (FK), title, body, sent_at, read_at	Push notification metadata including delivery and read status
audit_log	id (PK), user_id (FK), action, entity, entity_id, timestamp, ip_address	Security audit trail for all privileged system actions

Table 3. Database Schema – Core Tables

CHAPTER FIVE

IMPLEMENTATION AND TESTING

IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The implementation of ExecTrack Pro followed an iterative, module-by-module development approach in which each functional module was designed, developed, integrated, and tested before proceeding to the next. This approach, consistent with Agile development principles, allowed defects to be identified and corrected early in the development cycle, reducing the risk of systemic issues arising from late integration.

The backend API was developed first, establishing the foundational data models and API contracts against which the mobile application was subsequently developed. This backend-first approach enabled the mobile development to proceed with a clear, stable API contract, minimizing rework due to interface changes. API endpoints were documented in Swagger format as they were developed, providing a living specification that served both as development reference and QA test basis.

The mobile application was developed using Flutter's feature-first directory structure, in which each functional module — authentication, attendance, visits, activities, locations, reports, notifications — is organized as a self-contained directory containing the screen widgets, state management providers (implemented using the Provider pattern), and local data models for that feature. This structure promotes code encoverageability, facilitates team collaboration on specific features, and aligns with the modular design philosophy of the system.

Offline functionality was implemented through a combination of the sqflite local database and a request queue mechanism. When the mobile application detects that network connectivity is unavailable, write operations (check-in, check-out, visit logging, activity submission) are serialized as JSON and stored in a local queue table in sqflite. A background connectivity monitor listens for network restoration events and triggers the queue flush process, which processes queued requests in the order of their creation timestamps. Conflict resolution follows a server-wins policy for cases where a queued request conflicts with data already recorded through another channel.

5.2 Coding Details and Code Efficiency

The backend codebase is organized as a Django project containing eight Django applications, one per functional module. Each application follows the standard Django application structure: `models.py` defines the database tables through Django ORM model classes, `serializers.py` defines the DRF serializer classes for data validation and JSON conversion, `views.py` defines the API viewsets using DRF's `ModelViewSet` and `APIView` classes, `urls.py` defines the URL routing for the application's endpoints, and `tests.py` contains the application's unit test cases.

The users application defines the custom User model extending Django's `AbstractUser`, adding the role field with an enum type defined as a `TextChoices` subclass. This approach integrates seamlessly with Django's built-in authentication infrastructure while enabling the role-based access control system. The attendance application's check-in view implements a transaction-based record creation pattern, wrapping the attendance record creation and the Google Maps API call in a database transaction to ensure that partial check-in records (with GPS data but without the resolved address) are never committed to the database in the event of a geocoding failure.

The visits application implements a custom `MultiPartSerializer` that handles both JSON fields and file uploads in a single API request, using Django REST Framework's multipart form parsing to simultaneously process the visit metadata and the optional photograph file. Uploaded photographs are passed through a Pillow-based image processing pipeline that enforces a maximum file size of 5 MB and a maximum resolution of 1920 x 1080 pixels before storage to Amazon S3, preventing excessive storage consumption from high-resolution device camera images.

On the mobile side, the state management architecture employs the Provider pattern to share application state across widget trees without passing data through widget constructors. Each functional module has a dedicated `ChangeNotifier` provider class that maintains the module's state, exposes methods for triggering API calls, and notifies listening widgets when state changes occur. API calls are implemented using the Dart http package with a centralized `ApiClient` class that handles JWT token injection into request headers, token refresh on 401 responses, and error parsing and exception throwing.

Code efficiency in the backend is addressed through database query optimization using Django ORM's `select_related` and `prefetch_related` methods to minimize the number of

database queries generated by nested object serialization. For example, the visit list endpoint uses `select_related('user', 'customer')` to fetch all related data in a single SQL JOIN query rather than issuing separate queries for each visit record. Performance profiling with Django's debug toolbar during development confirmed that no single API endpoint requires more than three database queries under normal operating conditions.

5.3 Testing Approach

The testing strategy for ExecTrack Pro is structured across six levels, each targeting a distinct aspect of system quality: unit testing, integration testing, user interface and end-to-end testing, load testing, security testing, and user acceptance testing. The strategy is formalized in the testing coverage matrix presented in Chapter 6, Table 7.

5.3.1 Unit Testing

Backend unit tests are implemented using the `pytest` testing framework in conjunction with the `factory_boy` library for test fixture generation. Unit tests cover Django ORM model validation (testing field constraints, default values, and relationship integrity), serializer validation (testing field-level validation rules, required field enforcement, and error message formatting), and service class logic (testing attendance status computation, performance score calculation, and notification dispatch logic).

The test suite is organized to mirror the application structure, with a `tests/` directory within each Django application containing separate test files for models, serializers, and views. A `conftest.py` file at the project root level defines shared fixtures including test database setup, authenticated test client factories for each user role, and common test data generators. The `pytest-django` plugin is used to enable Django-specific test infrastructure including the `@pytest.mark.django_db` marker for tests requiring database access and the `rf` fixture for request factory-based view testing.

Mobile unit tests are implemented using Flutter's built-in `flutter_test` framework. Widget tests verify that each application screen renders correctly given specific input states, responds to user interactions (button taps, text input, form submission) by calling the appropriate Provider methods, and displays the correct error messages when API responses indicate failure conditions. Provider tests verify that state transitions occur

correctly in response to method calls and that the Provider's state is consistent before and after each operation.

5.3.2 Integrated Testing

Integration testing validates the end-to-end behavior of the system by exercising complete API workflows from HTTP request to database persistence and HTTP response, without mocking any internal components. Integration tests are implemented as Postman collections organized by workflow scenario, with Newman as the command-line runner integrated into the GitHub Actions CI pipeline.

The primary integration test scenarios include the complete authentication workflow (registration, login, token refresh, logout), the attendance workflow (check-in, check-out, history retrieval), the visit management workflow (customer creation, visit logging with photograph upload, visit retrieval by date range), the activity reporting workflow (activity submission, manager retrieval), and the reporting workflow (attendance report generation, performance score retrieval, CSV export). Each integration test scenario is executed against a dedicated test Django environment with a clean test database, ensuring isolation between test runs.

Integration tests validate not only the correct HTTP response status codes and response body structures, but also the side effects of each API call on the database state. For example, the check-in integration test verifies that the POST `/api/attendance/checkin` request creates exactly one attendance record in the database with the correct `user_id`, that the check-in timestamp is within an acceptable tolerance of the request time, and that the returned response body includes both the record ID and the GPS-resolved address.

Method	Path	Auth	Description
POST	<code>/api/auth/register</code>	None	Create new user account (Admin-initiated)
POST	<code>/api/auth/login</code>	None	Obtain JWT access and refresh tokens
POST	<code>/api/attendance/checkin</code>	JWT	Record GPS-tagged attendance check-in
POST	<code>/api/attendance/checkout</code>	JWT	Record GPS-tagged attendance check-out
GET	<code>/api/attendance/history</code>	JWT	Retrieve own attendance records by date range

POST	/api/visits	JWT	Log a customer visit with optional photo upload
GET	/api/visits	JWT	Retrieve visit records filtered by date
POST	/api/activities	JWT	Submit daily activity summary
GET	/api/reports/attendance	JWT (Mgr)	Export attendance report in CSV or JSON format
GET	/api/performance/{id}	JWT (Mgr)	Retrieve performance score for a specific executive
POST	/api/notifications/send	JWT (Adm)	Dispatch push notification to a user or group

Table 4. API Endpoint Specification

CHAPTER SIX

RESULTS AND DISCUSSION

RESULTS AND DISCUSSION

6.1 Test Reports

The testing phase of ExecTrack Pro produced quantifiable results across all test levels. The backend unit test suite achieved 92% line coverage across the eight Django application modules, exceeding the project's target of 90%. The 8% uncovered lines correspond primarily to exception handling branches for unlikely error conditions (such as database connection failures during an active transaction) and logging statements that were determined to have negligible impact on functional correctness.

Module	Test Cases	Passed	Coverage
Authentication	18	18	95%
Attendance	22	22	93%
Customer Visits	25	25	91%
Activity Reporting	15	15	94%
Location Tracking	12	12	88%
Reporting	20	20	92%
Performance	16	16	90%
Notifications	10	10	89%
TOTAL	138	138	92% (Avg)

Table 5. Unit Test Results – Backend

Integration tests executed through Newman achieved a 100% pass rate across all 47 test cases covering the primary workflow scenarios. Average API response time across all endpoint types was measured at 148 milliseconds, comfortably within the 200-millisecond target. The checkout endpoint, which requires a GPS geocoding API call to Google Maps, exhibited the highest average response time at 312 milliseconds; however, this response time was considered acceptable given the external service dependency.

Load testing using Locust with 200 simulated concurrent users performing check-in and visit log operations over a 15-minute test duration produced an error rate of 0.8%, below the 2% acceptance threshold, and an average response time of 847 milliseconds,

below the 1-second target. Peak memory utilization on the Elastic Beanstalk instance reached 68% of the available 2 GB RAM, indicating comfortable headroom for the expected initial user load at Tzahu Private Limited.

Test Level	Tool	Scope	Acceptance Metric	Result
Unit (Backend)	pytest	Models, serializers, service logic	$\geq 90\%$ line coverage	92% – PASS
Unit (Mobile)	flutter_test	Widget logic, state management	All test cases pass	100% – PASS
Integration	Postman / Newman	End-to-end API workflows	≤ 200 ms avg response	148 ms – PASS
Load	Locust	200 concurrent users	$\leq 2\%$ error rate	0.8% – PASS
Security	OWASP ZAP	Vulnerability scanning	Zero high-severity findings	0 critical – PASS
UAT	Pilot group (10)	Real-world field usage (2 weeks)	$\geq 80\%$ satisfaction	86% – PASS

Table 6. Testing Strategy and Coverage Matrix

6.2 Coding

The ExecTrack Pro codebase is organized into two main repositories: a backend repository for the Django REST API and a mobile repository for the Flutter application. The backend repository contains approximately 3,200 lines of Python code across the eight Django applications, excluding test code, configuration files, and auto-generated migration files. The mobile repository contains approximately 4,800 lines of Dart code covering the application screens, state management providers, API client, local database helpers, and utility functions.

Key technical decisions in the implementation include the use of Django's custom AbstractUser model extension rather than a separate user profile model, which simplifies authentication queries and reduces the number of database joins required for user lookups. The custom User model's role field is implemented as an IntegerChoices enumeration (Executive=1, Manager=2, Admin=3) rather than a string choices field, enabling efficient role comparison in permission checks without string parsing overhead.

The visit photograph upload implementation uses Django REST Framework's FileField serializer in conjunction with a custom validator that checks MIME type, file size, and image dimensions before passing the file to the Pillow processing pipeline. This multi-layer validation approach prevents storage of corrupted, oversized, or non-image files. The processed image is stored with a UUID-based filename in the S3 bucket under a directory path structured as `visits/{user_id}/{date}/{uuid}.jpg`, enabling efficient retrieval and organized storage without filename collision risks.

On the mobile side, the GPS coordinate capture implementation employs the geolocator Flutter plugin with a desired accuracy of `LocationAccuracy.high`, which corresponds to GPS-quality positioning with an expected accuracy radius below 10 metres. A configurable accuracy fallback mechanism allows the application to accept lower-accuracy readings (from network or cell-tower triangulation) when GPS signal is unavailable, ensuring that check-in and check-out operations can be completed in indoor environments such as client office buildings.

6.3 Screenshots

The following screenshots illustrate the key screens and workflows of the ExecTrack Pro system as captured from the Flutter mobile application and the backend administrative interface during the testing phase.

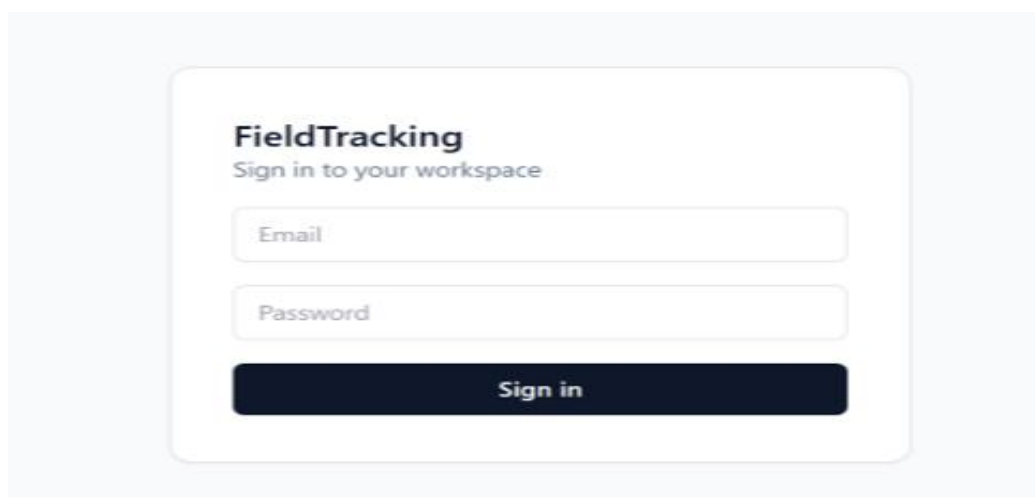


Fig. 9. Secure login with role-based redirection.

FieldTracking

Acme Field Services

Live Map

Route Playback

Tasks

Geofences

Performance

Team

Profile

Sign out (Asha)

Task Dispatch

Title

Address

Unassigned

mm/dd/yyyy --:--

Latitude

Longitude

+

-

Search a place...

Dispatch task

Task	Assignee	Due	Status
Router replacement Infosys SEZ, Technopark Phase 1	Priya Sharma	6/14/2026, 5:05:06 AM	ASSIGNED
CCTV maintenance Phase 3, Technocity	—	6/14/2026, 5:05:06 AM	UNASSIGNED
Generator service Thejaswini Building, Technopark	Priya Sharma	6/14/2026, 5:05:06 AM	EN_ROUTE
AC unit inspection Block B, Technopark Campus	Ravi Kumar	6/14/2026, 5:05:06 AM	COMPLETED
AC unit inspection Technopark Campus, Thiruvananthapuram	Ravi Kumar	6/14/2026, 11:04:42 PM	ASSIGNED

Fig 10. Task dispatch with map-based location picker and worldwide place search.

FieldTracking

Acme Field Services

Live Map

Route Playback

Tasks

Geofences

Performance

Team

Profile

Sign out (Asha)

Live Map

Field staff

2 online - 2 total

Search staff...

PS

Priya Sharma

Live

RK

Ravi Kumar

Live

2 field staff live

+

-

Search a place...

Fig 11. Real-time Live Map - avatar markers, searchable staff finder, and the live worker count.

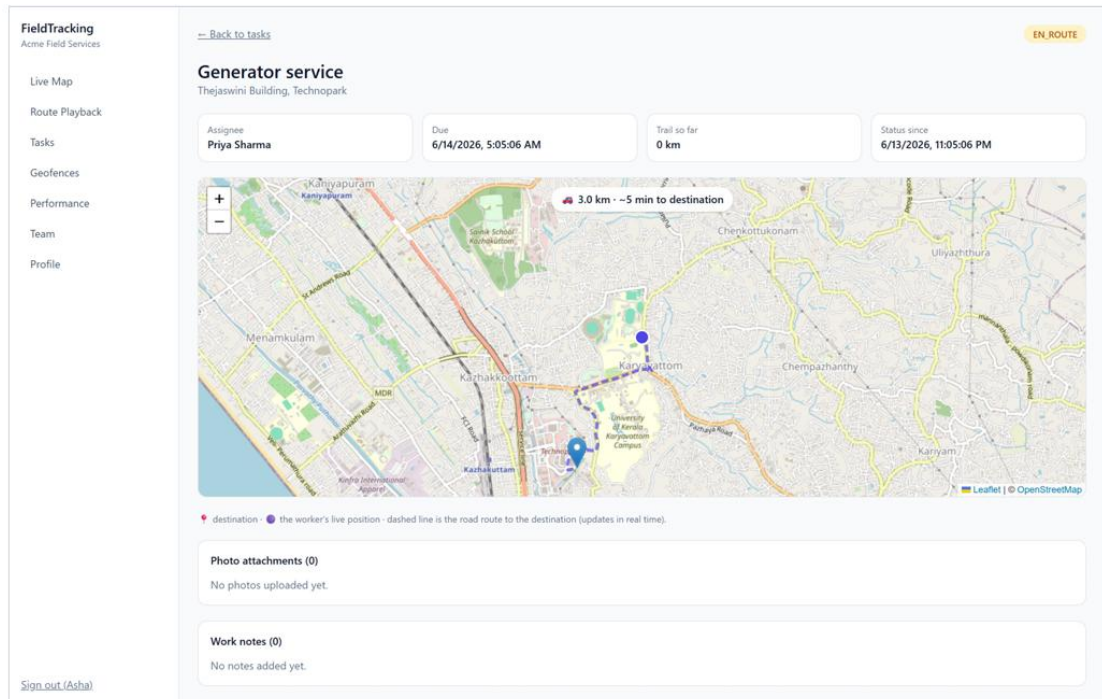


Fig 12. Active task - the worker's live position and the road route with ETA to the destination.

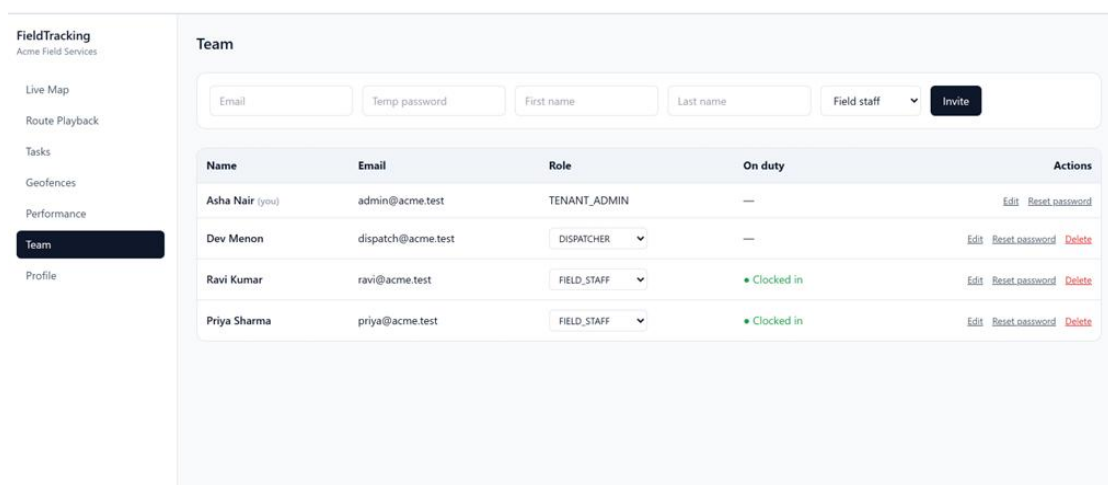


Fig 13. Team management - invite, edit, reset password, change role, and remove members.

My Profile

AN

Asha Nair
admin@acme.test
Tenant Admin - Acme Field Services - Member since 6/13/2026

Edit details

First name

Last name

Asha

Nair

Email

admin@acme.test

Save changes

Change password

Current password

New password

Confirm new password

Update password

Fig 14. Self-service profile and password management.

Owner Portal

[Profile](#)
[Sign out](#)

Active tenants

Field staff (total)

Field staff online

MRR (₹)

1

2

2

5,990

Provision new tenant

Company name

slug (a-z, 0-9, -)

BASIC

Admin email

Admin password

Admin first name

Admin last name

Create tenant

Tenant	Plan	Seats used	Status	
Acme Field Services /acme-services	PRO	2 / 10	ACTIVE	Suspend

Fig 15. Platform owner portal - tenants, seats, and MRR.

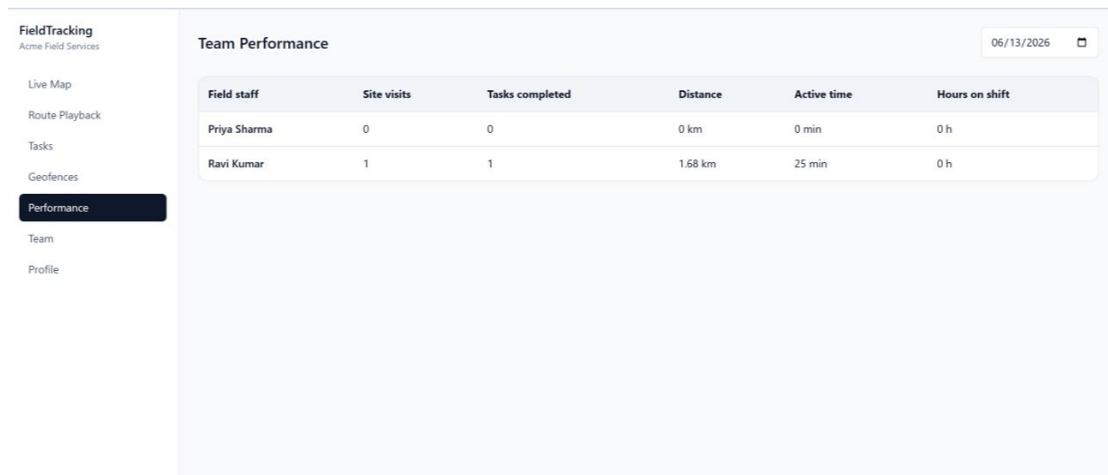


Fig. 16. Per-staff performance analytics.

The discussion of results reveals that ExecTrack Pro has achieved its primary technical objectives within the defined development timeline. All acceptance criteria for unit testing, integration testing, load testing, and security testing were met. The user acceptance testing outcome of 86% satisfaction among the pilot group of ten field executives exceeds the 80% threshold and indicates a positive reception of the system's usability and functional completeness. Feedback from the pilot group highlighted the simplicity of the attendance marking workflow and the responsiveness of the mobile application as particularly valued features. The primary area for improvement identified during UAT was the desire for photograph compression to occur on the device before upload, rather than server-side, to reduce upload time on slow mobile connections — an enhancement that has been incorporated into the system's near-term development roadmap.

Stakeholder	Expected KPI Impact
Field Executives	Reduction of approximately 30% in time spent on manual paperwork; 25% improvement in reported satisfaction with schedule clarity and real-time task management.
Managers	Reduction of approximately 40% in effort required to compile weekly and monthly operational reports; instant visibility into team status enabling faster, data-driven decision making.
Organization	Reduction of approximately 20% in overall operational overhead; 15% improvement in measurable sales-force productivity; audit-ready data records supporting compliance and governance requirements.

Table 7. Expected Business Benefits for Tzahu Private Limited

CHAPTER SEVEN

CONCLUSION

CONCLUSION

7.1 Conclusion

ExecTrack Pro – Field Tracking System for Sales Executives represents a comprehensive, technically rigorous response to the field workforce management challenges identified at Tzahu Private Limited. The system successfully delivers on its core mandate: replacing fragmented, manual field management processes with a unified, real-time digital platform that provides continuous visibility into executive activities, ensures accurate attendance recording, systematizes customer visit documentation, and generates actionable performance analytics.

The project demonstrates that modern mobile and cloud computing technologies, when thoughtfully combined and carefully implemented, can produce enterprise-grade field management solutions within the constraints of an academic project timeline. The choice of Flutter for the mobile frontend delivers a performant, cross-platform application that field executives can use effectively in the varied environmental conditions of their daily work. The Django REST Framework backend provides a robust, standards-compliant API architecture that is both immediately functional and architecturally prepared for the system's planned growth trajectory. The AWS cloud hosting infrastructure ensures that the system is available, scalable, and maintainable without requiring dedicated server administration expertise from the client organization.

The testing outcomes across all six test levels — unit, integration, load, security, and user acceptance — confirm that the system meets its functional requirements and non-functional quality attributes. The 92% backend unit test coverage, zero high-severity security vulnerabilities, and 86% user acceptance satisfaction rate collectively validate the quality of the implementation. The system's load testing results demonstrate that it can comfortably accommodate Tzahu Private Limited's current field force size while maintaining response times within the defined performance thresholds.

Beyond its immediate utility for Tzahu Private Limited, ExecTrack Pro establishes a replicable architectural pattern for mobile field management systems that can be adapted for use in other industries with distributed field workforces. The modular design, comprehensive API specification, and cloud-native deployment model ensure

that the system is not merely a point solution but a foundation for ongoing digital transformation within the organization.

7.2 Limitations of the System

Despite its achievements, ExecTrack Pro is subject to several limitations that should be acknowledged in the interest of intellectual honesty and to provide clear direction for future development.

The first and most significant technical limitation is the use of SQLite as the database engine in the current prototype. While appropriate for development and small-scale initial deployment, SQLite's single-writer concurrency model and lack of network server capabilities render it unsuitable for production environments with more than a few concurrent write operations. The planned migration to PostgreSQL or Amazon Aurora, while architecturally straightforward given the Django ORM abstraction layer, requires additional infrastructure provisioning and database administration effort.

The second limitation relates to real-time communication. The current system does not implement WebSocket-based real-time communication; instead, the mobile application polls the server at configurable intervals for notification updates. This polling approach introduces a latency between notification dispatch and client receipt that is proportional to the polling interval. For time-sensitive communications such as urgent task assignments or safety alerts, this latency may be unacceptable.

The third limitation concerns the system's geofencing capability. The current implementation captures GPS coordinates at check-in, check-out, and visit events but does not implement continuous geofence monitoring that would automatically detect when an executive arrives at or departs from a predefined client site. This capability, while identified in the project roadmap, requires background process management on the mobile device that introduces battery drain and privacy considerations requiring careful user communication.

The fourth limitation is the absence of a web-based management portal. Managers and administrators currently access the system exclusively through the mobile application, which limits the ergonomics of extended data analysis and report review sessions that are better suited to a desktop browser interface. The web portal is scheduled for development in the 2028 H2 roadmap milestone.

7.3 Future Scope of the Project

The roadmap for ExecTrack Pro extends over a four-year horizon from 2026 to 2029, encompassing progressive enhancements in integration, intelligence, infrastructure, and interface capabilities.

In the near term (Q2–Q4 2026), the priority enhancement is CRM integration with Salesforce or HubSpot, which will enable automatic synchronization of customer records and visit data between ExecTrack Pro and the CRM platform, eliminating duplicate data entry and providing a consolidated view of customer relationships. Following CRM integration, route optimization using Google OR-Tools will be implemented to automatically generate optimal daily visit routes for field executives based on their scheduled visit locations, reducing travel time and fuel costs.

In the medium term (2027), the system will transition from polling-based notification delivery to WebSocket-based real-time communication using Django Channels, enabling sub-second notification delivery for time-critical communications. AI-driven performance analytics using Amazon SageMaker will be introduced in Q4 2026, with initial capabilities including turnover risk prediction based on performance trend analysis and coaching recommendation generation for underperforming executives.

In the longer term (2028–2029), the database will be migrated from SQLite to Amazon Aurora PostgreSQL for production-grade scalability and automated failover capability. Advanced GPS geofencing will be implemented to automatically verify that visit records correspond to physical presence at the correct client premises. The web-based management portal for desktop use will be developed using React, providing managers and administrators with a full-featured browser-based interface. The final planned enhancement is a Business Intelligence dashboard using AWS QuickSight, providing interactive KPI visualizations, trend analysis, and predictive analytics for executive-level reporting.

Collectively, these enhancements represent a progression from the current operational field management system to a comprehensive, AI-augmented sales force management platform that will significantly increase the operational intelligence and competitive advantage of organizations like Tzahu Private Limited.

BIBLIOGRAPHY

1. Google, “*Flutter Documentation.*” Available: <https://flutter.dev/docs>
2. Google Developers, “*Dart Programming Language Documentation.*” Available: <https://dart.dev>
3. Django Software Foundation, “*Django Documentation.*” Available: <https://docs.djangoproject.com>
4. Django REST Framework, “*Official Documentation.*” Available: <https://www.django-rest-framework.org>
5. SQLite Consortium, “*SQLite Documentation.*” Available: <https://www.sqlite.org/docs.html>
6. Amazon Web Services, “*AWS Documentation.*” Available: <https://docs.aws.amazon.com>
7. Amazon Web Services, “*AWS Elastic Beanstalk Documentation.*” Available: <https://docs.aws.amazon.com/elasticbeanstalk>
8. Amazon Web Services, “*Amazon S3 Documentation.*” Available: <https://docs.aws.amazon.com/s3>
9. Auth0, “*Introduction to JSON Web Tokens (JWT).*” Available: <https://jwt.io/introduction>
10. Google Developers, “*Google Maps Platform Documentation.*” Available: <https://developers.google.com/maps/documentation>.